

Title: Resolving JML ETL Incidents and Migration Strategy to Azure/Entra

1. Executive Summary

This document outlines the approach to investigating and resolving incidents affecting the Council's Joiner-Mover-Leaver (JML) legacy ETL pipeline, which currently prevents data from being imported into the Learning Management System (LMS). As a new member of the data integration team, I will approach this systematically by understanding the existing environment, addressing my knowledge gaps, identifying root causes, and developing a sustainable solution that includes modernizing the ETL through migration to Azure/Entra.

2. Understanding and Assessing the Root Cause

Step 1: Initial Investigation

- **Review incident tickets:** Collect information from historical incidents and identify recurring failure patterns, time of occurrence, and data volumes involved.
- **Examine ETL job logs:** Look for error codes, timestamps, and execution durations to identify the specific ETL phase (Extract, Transform, or Load) where the issue arises.
- **Validate data sources:** Check for missing or malformed input files, database connection issues, or data format mismatches.
- **Engage stakeholders:** Conduct quick interviews with LMS administrators, IT operations, and data owners to confirm if there were any recent changes to source systems, schemas, or authentication configurations.

Step 2: Environment Analysis

- Assess the **infrastructure setup:** Identify whether the ETL jobs are hosted on an on-premise SQL Server, Windows service, or a scheduled batch job.
- Evaluate **data flow dependencies:** Determine if multiple upstream systems (e.g., HR, payroll, or Active Directory) feed data into the JML ETL pipeline.
- Review **system resources:** Monitor CPU, memory, and disk I/O utilization during ETL execution to detect performance bottlenecks.
- Verify **connectivity and credentials:** Ensure that connections to data sources are valid and not impacted by password expirations or network firewalls.

Step 3: Root Cause Identification

- Correlate all gathered evidence to identify the source of the issue.
- Recreate the problem in a test environment to confirm the failure point.

- Document every hypothesis and verify through controlled testing before implementing changes in production.

Outcome: A validated diagnosis of the issue, with evidence and confidence in the identified root cause.

3. Detailed Real-Time Scenario Example

Scenario 1: Credential Expiry Causing ETL Failure

Context:

During a nightly JML ETL job run, multiple user records failed to sync from the HR system into the LMS. The monitoring dashboard flagged the issue, and several new staff members were unable to access mandatory training modules. This impacted onboarding and compliance activities across departments.

Investigation Process:

1. The initial alert came from the ETL scheduler, which showed multiple job failures with the message `SQLSTATE[08001]: [Microsoft][ODBC Driver] Cannot open connection`.
2. Checking the SQL Agent logs revealed that the job failed during the data extraction phase from the HR database.
3. Upon further analysis, it was found that the ETL used a static database account whose password had expired following an automated security policy update.

Resolution:

- Temporarily reset the service account credentials and updated the connection string for the ETL job.
- Moved sensitive credentials into **Azure Key Vault** to avoid storing them in plain-text configuration files.
- Implemented an **automated credential rotation process** tied to Azure Active Directory policies.

Preventive Action:

- Introduced automated alerting that triggers when credentials approach expiration.
- Conducted a workshop with IT Security to align the ETL authentication mechanism with corporate standards.

Result:

Normal data flow was restored, ensuring all new employees could access the LMS the same day. Post-fix monitoring showed no further failures related to authentication.

Scenario 2: Transformation Logic Error Leading to Data Mismatch

Context:

A batch of mover records (employees changing departments) was not reflected correctly in the LMS. While the data extraction worked fine, the transformation logic failed to map department codes due to a schema change in the HR system.

Investigation Process:

1. The ETL logs indicated a truncation error when applying transformation rules on the `Department_Code` field.
2. A review of the HR database schema showed that department codes were recently expanded from 5 to 10 characters.
3. The ETL transformation logic still expected a 5-character field and truncated data during processing.

Resolution:

- Updated the transformation scripts to accommodate the new schema structure.
- Added validation scripts to cross-check schema compatibility before ETL execution.
- Coordinated with the HR system team to establish a **schema change notification process**.

Preventive Action:

- Introduced schema validation as a pre-run ETL check.
- Integrated CI/CD pipelines to test ETL transformations upon every version update.

Result:

Data accuracy improved significantly, and no further mismatches occurred after the fix. The new process also prevented unnoticed schema drift between systems.

Scenario 3: Migration Impact on Performance (Pre-Migration Pilot)

Context:

While testing the migration of the ETL pipeline from on-premise SQL Server to Azure Data Factory (ADF), developers noticed a delay in job execution times. The cloud-based data transfer took 30% longer than the legacy on-prem job.

Investigation Process:

1. Analyzed data transfer patterns and found that network latency and lack of parallelization were key contributors.
2. The on-prem ETL used local file staging, while ADF relied on blob storage for intermediate data handling.

Resolution:

- Enabled **parallel copy activity** in ADF pipelines to improve throughput.
- Optimized ADF integration runtime to use a dedicated compute cluster.
- Cached frequently used reference data in Azure SQL to reduce redundant queries.

Preventive Action:

- Conducted multiple load tests with different batch sizes.
- Established baseline performance benchmarks for all future ETL migrations.

Result:

The performance improved by 40%, making ADF a viable and scalable option for production workloads.

4. Addressing Knowledge Gaps

As a new team member without prior experience on the JML ETL, the following steps will ensure rapid knowledge acquisition:

1. **Documentation Review:** Study existing ETL architecture diagrams, database schemas, and historical issues.
 2. **Mentorship:** Pair with experienced engineers for code walkthroughs and environment orientation.
 3. **Sandbox Environment:** Practice running ETL pipelines independently to understand dependencies.
 4. **Continuous Learning:** Take Microsoft Learn modules on Azure Data Factory, SQL optimization, and Entra integration.
 5. **Knowledge Sharing:** Document findings and lessons learned in a shared Confluence space.
-

5. Preventing Recurrence: Operational and Process Improvements

- **Implement Monitoring and Alerts:** Use Azure Monitor and Application Insights to track ETL job metrics, execution time, and data anomalies.
- **Adopt Data Quality Frameworks:** Introduce pre-load and post-load validation to ensure accuracy.

- **Enhance Change Control:** Introduce Git-based version control for ETL scripts and enforce peer review workflows.
 - **Conduct Post-Mortems:** Hold retrospectives for major incidents to identify gaps and refine processes.
-

6. Migration Strategy: On-Premises ETL to Azure/Entra Environment

1. **Assessment:** Evaluate all ETL jobs, data sources, and dependencies. Identify components suitable for Azure Data Factory (ADF) migration.
 2. **Architecture Design:** Define the target architecture using ADF, Azure SQL, Data Lake, and Entra ID for authentication.
 3. **Pilot Migration:** Select one low-risk ETL job for pilot migration to validate the pipeline design.
 4. **Full Migration:** Incrementally migrate jobs, ensuring data validation and parallel test runs.
 5. **Optimization:** Enable monitoring, autoscaling, and CI/CD for ADF pipelines.
 6. **Post-Migration Review:** Evaluate performance, cost, and compliance to ensure long-term maintainability.
-

7. Conclusion

Through detailed incident investigation, continuous knowledge development, and operational maturity, recurring JML ETL issues can be eliminated. Migrating to Azure/Entra provides scalability, enhanced security, and automation—ensuring that the Council's data integration processes remain resilient and future-ready.
